CS4040/B — 自然语言处理 — 期末报告

Yelp 评论分析：从 Baseline 到商业智能 Agent

一个把回答 NLP 问题、给出工程决策变 Agentic RAG 系统

GitHub: github.com/YuhanWang33/Yelp-Project

项目概览

本项目对 Yelp 餐厅点评数据（约 25 万条）构建了一个四层递进的 NLP 实践流水线，每一层回答前一层回答不了的问题：从简单的词袋分类出发，到深度模型对比，再上升到可解释性多维方面分析，最终落地成一个能对业务指标做出建议的 Agentic 检索增强系统。

Stage	Baseline 回答	不回答的问题
1	TF-IDF + Logistic Regression	哪个方面（服务、菜品、环境）在拉低评分？
2	Sentence BERT + Logistic Regression	模型学到的是语义还是表面词频？
3	BERT（微调）	模型能不能解释它的预测？
4	Agentic RAG + Langchain Agent	基于这些洞察，我应该给商家建议什么？

任务定义	BERT 表现	关键结论
二分类（正面/负面，忽略中性）	98.56%	简单任务，TF-IDF 已经很强
三分类（正面/中性/负面）	92.60%	中性类是瓶颈，F1 比正面低 ~6 个点
五分类（1–5 星）	67.98%	相邻评级容易混淆，MAE 0.42

核心洞察：特征工程 → 固定预训练表示 → 端到端 Transformer 微调 — 大模型能提取语义，但模型不会解释它做出决策的原因。

第一部分 — Baseline 对比

在 5 分类数据集上，对比三种特征工程方法的表现：词袋（Bag of Words）、

模型	Accuracy	Macro F1
TF-IDF + Logistic Regression	56.92%	56.56%
Sentence BERT + Logistic Regression	53.25%	52.89%
BERT（微调）	**62.50%**	**62.50%**

为什么选择 BERT 效果最好？词袋模型仅以文本表示因此对语义不敏感，每一个词都会被独立对待。Sentence BERT 冻结了预训练权重，只学习一个线性分类器——它的表现受限于预训练表示本身的质量。BERT 通过在 Yelp 数据上微调，让整个网络学习到对情感分析有用的表示。

为什么 Sentence BERT 表现最差？冻结 BERT 的权重会丢失大部分语义信息，因为预训练目标是通用的，并不针对情感分析。BERT 微调则保留了这些信息，并将其重新对齐到情感分类任务上。

结论：在这个数据集上，BERT 微调是性价比最高的方案，但其表现受限于预训练语料质量。

第二部分 — BERT 错误分析

在以下所有指标上，BERT 微调模型（checkpoint，fine-tuning 前）：

指标	数值
Backbone 模型	distilbert-base-uncased，hidden_size = 768，num_layers = 6
任务设置	二分类（Drop 中性），3 个类别，5 个类别
训练模式	Classification（交叉熵）、Regression（MSE）
学习率	1e-5、2e-5
训练轮数	2、4、16
训练样本 (Epochs)	3、4、2
总组合数	**3 × 2 × 2 × 2 × 3 = 72**
总样本数	**216**

每组实验在固定种子 (seed = 42) 下运行，保证数据顺序、初始化、dropout 等随机性可复现。

实验	最终 Accuracy	最终 Macro F1	Off-by-1 错误率
DistilBERT-base-uncased	96.54%	96.53%	—
BERT-base-uncased	97.07%	97.06%	—
RoBERTa-base	**97.82%**	**97.81%**	**0.3968**

5 分类 — 测试集

模型	测试 Accuracy	测试 Macro F1	Off-by-1 错误率
DistilBERT-base-uncased	65.42%	65.36%	97.25%
BERT-base-uncased	66.75%	66.36%	97.49%
RoBERTa-base	**68.56%**	**68.49%**	**96.19%**

Off-by-1 准确率（指标定义）：衡量模型预测结果与真实标签相差不超过 1 个等级的比例。例如，预测 4 vs 真实 5 算作 Off-by-1，但预测 1 vs 真实 5 不算。这个指标对于评估序数分类任务尤为有用——在 1 和 5 之间随机猜测会产生较高的 Macro F1，但 Off-by-1 会暴露出这种错误模式。

为什么 RoBERTa 胜过 BERT 和 DistilBERT？RoBERTa 在预训练时采用了更大的 batch size、更长的 Masking 策略、并使用了更大规模的数据进行训练。在相同超参数设置下，这些修改带来了稳定的表现提升，DistilBERT 则因蒸馏过程而牺牲了部分表示能力。

为什么 LR-1e-5 和 Epoch = 2 是最佳配置？较小的学习率会导致欠拟合，尤其是对预训练模型——模型会保留通用语言知识，但无法学习任务特定的调整。2 个 epoch 对于 fine-tuning 来说通常是足够的，超过这个数值后，验证集指标开始出现过拟合。

最终配置：在 5 分类任务上，RoBERTa-base，LR=1e-5，batch=4，epochs=4，seed=42，checkpoint 在测试集上的表现：

第三部分 — 大语言模型评测与对比

对两个指令微调大语言模型，在同一评估协议下，分别以零样本和少样本方式进行测试：

Qwen2.5-7B-Instruct	DeepSeek-R1-Distill-Qwen-14B
第一轮：零样本基线	第一轮：零样本基线
第二轮：LoRA 微调（QLoRA / bf16）	第二轮：思维链提示（few-shot）
	第三轮：LoRA 微调 + 思维链训练数据

训练配置详情

Qwen2.5-7B-Instruct	MPNS (A100 80 GB)
量化	4-bit NF4 + 双量化（bfloat16 计算）
LoRA 秩	16
LoRA alpha	32
LoRA dropout	0.05
目标模块	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj（所有线性层）
学习率	2e-4
训练轮数	2
有效批大小	4 × 4 = 16
学习率调度	Cosine，warmup ratio 3%
训练样本数	2000（随机 10240）
验证样本数	200
最大序列长度	512
精度	bf16
梯度检查点	开启
随机种子	42

第一步：生成思维链	在训练集上运行 DeepSeek，记录能生成正确答案的样本作为训练数据
第二步：LoRA 微调	使用指导格式在思维链数据上训练，保留推理能力

DeepSeek 思维链 LoRA 超参数：

参数	值
LoRA 秩	16
LoRA alpha	32
目标模块	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
学习率	**5e-5**
训练轮数	2
有效批大小	4 × 4 = 16
训练样本数	**1024**
验证样本数	200

Assistant 回答的训练格式：

```
<think>
{step_1_reasoning}
</think>
Final Label: {1 | 2 | 3 | 4 | 5}
```

训练数据以“先思考后输出”的格式学习：模型学习在给出最终答案之前，首先生成结构化的推理过程。

DeepSeek 推理时格式 — 第二轮消息

参数	值
推理模式	贪婪解码（temperature = 0）
最大新生成 token 数	512（为推理链留出空间）
输出解析	从生成文本中提取最后一个合法标签
解析失败回退	默认为 3（中性）

零样本提示（两轮实验共用）

```
System: You are a sentiment classifier.
Reply with exactly one word: 1, 2, 3, 4, or 5.
No other text, no explanation, no punctuation.
Rating scale: 1 = very negative, 2 = negative, 3 = neutral, 4 = positive, 5 = very positive.
User: Review: "{text}"
Rating:
```

思维链提示（两轮实验共用）

```
System: You are a sentiment classifier.
Reply with exactly one word: 1, 2, 3, 4, or 5.
No other text, no explanation, no punctuation.
Rating scale: 1 = very negative, 2 = negative, 3 = neutral, 4 = positive, 5 = very positive.
User: Review: "{text}"
Rating:
```

DeepSeek 思维链 LoRA 训练配置

参数	值
量化	4-bit NF4 + 双量化（bfloat16 计算）
LoRA 秩	16
LoRA alpha	32
LoRA dropout	0.05
目标模块	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj（所有线性层）
学习率	2e-4
训练轮数	2
有效批大小	4 × 4 = 16
学习率调度	Cosine，warmup ratio 3%
训练样本数	2000（随机 10240）
验证样本数	200
最大序列长度	512
精度	bf16
梯度检查点	开启
随机种子	42

保存位置	每个检查点保存一份 adapter 权重，以便事后对比各个 checkpoint 的表现
硬件	A100 80 GB
评估	与 Qwen 零样本对比，保证公平可比

结果图谱（max_tokens=128，零样本）

模型	准确率	解析失败率
Qwen-7B-Instruct	56.14%	0.00%
Qwen2.5-7B-Instruct	**LoRA 微调（SFT）**	**98.55%**
DeepSeek-R1-14B	零样本	80.04%
DeepSeek-R1-14B	思维链（CoT）	79.41%
DeepSeek-R1-14B	思维链 + LoRA 微调（SFT）	98.42%

5 分类 — 测试集

模型	准确率	解析失败率
Qwen-7B-Instruct	56.14%	0.00%
Qwen2.5-7B-Instruct	零样本	67.36%
Qwen2.5-7B-Instruct	LoRA 微调（SFT）	67.73%
DeepSeek-R1-14B	思维链（CoT）	61.56%
DeepSeek-R1-14B	零样本	55.60%

为什么 Qwen 零样本表现最好？指令遵循能力强，能在不微调的情况下直接输出合法格式。思维链模型倾向生成长篇推理，在 max_tokens=128 的限制下会被截断，导致解析失败率高达 80%。

为什么 DeepSeek 思维链 LoRA 表现不如 Qwen？尽管经过微调，模型仍会生成冗长的推理过程，导致部分输出被截断。这表明问题不在于模型能力，而在于输出格式与任务要求的匹配程度。

为什么 DeepSeek 零样本表现最差？模型倾向输出长篇推理，而非直接给出标签。这种行为在开放式问答中是合理的，但在结构化分类任务中会成为主要失败模式。

为什为 DeepSeek 微调模型推理能力的提升？思维链训练数据（1024 条）仅覆盖一小部分训练集，模型学习到了一种通用的推理模式，但这种模式并未完全对齐任务要求。

结论：Qwen 在指令遵循方面表现最优，LoRA 微调以牺牲格式灵活性为代价，换取了更高的准确率。思维链模型在这个任务上并不具备优势——推理过程会干扰结构化输出。

第四部分 — 商业智能 Agent

第四部分将任务从分类升级为 NLP 评估数据的问答系统。构建了两层系统：用于检索的 BM25 索引，用于生成答案的工具调用代理。

数据规模

第一层 — RAG Baseline（检索层）	第二层 — Full Agent（langchain-based 工具）
1 get_business_stats（按商家统计聚合）	Qwen2.5-7B（本地 Ollama）
2 search_by_business_id_and_aspect（多维过滤）	工具调用 Agent
3 semantic_evidence（语义检索）	工具调用 Agent
	结构化输出解析器
	自然语言工具调用回退

Full Agent 内部的 ReAct 循环结构图，直观呈现内部工具调用流程：

技术栈

langchain_core_tract_agent	Qwen2.5-7B via Ollama	FAISS IndexFlatIP	all-MiniLM-L6-v2	RoBERTa-base 分类器	Langchain + langchain-ollama	Gradio

问题写回字段

- 5 次数 字段 — **4,5,2,1 个文本块**（按句子切分，每块 ≤180 词）
- 使用 `all-MiniLM-L6-v2` 编码（384 维，L2 归一化，内积等价于余弦相似度）
- FAISS `IndexFlatIP` 精确最近邻搜索
- 推断为预测结果，仅在实际标签缺失时入子集预测结果，避免小数据集标签稀疏问题

评估协议

20 条问题 × 4 种配置 × 3 个随机 seed = 60 组实验，人工给 5 个维度上评分（满分 2，满分 10 分）：

指标	含义
回答相关性	“回答与问题相关吗？”
方面分析	“回答与所询问方面（服务/菜品/价格/氛围/等待）相关吗？”
数据支撑	“回答内容，确实出自检索到的证据吗？”
商业洞察	“Yelp 评论中，哪些结构化指标值得关注？”
可操作性	“回答能帮商家做出具体决策吗？”

评测结果

配置	正确率	证据覆盖率	可解析率	工具调用率	延迟	成本
Direct LLM	0.25	0.00	0.00	0.00	1.70	1.95
RAG Baseline	**0.55**	**1.00**	**1.00**	**0.95**	1.61	0.90
Full Agent	1.05	1.15	1.15	1.00	4.02	2.10

RAG Baseline（0.55/10）：固定检索流程稳定可靠，证据引用完整，结构化解析 100%，但无法调整检索策略以应对复杂问题。

Full Agent（1.05/10）：工具调用率 100%，证据覆盖率 115%，可以多次检索并交叉验证，但延迟是 RAG 的 2.5 倍，成本是 2.3 倍。

Direct LLM（0.25/10）：无法访问证据，回答与问题相关，但缺乏数据支撑，结构化解析率 0%，可操作性极低。

结论：在这个任务上，结构化检索的收益远大于代理推理的收益。小模型（7B）在工具调用上表现良好，但多轮推理的边际收益递减。

结论与反思

- **RAG 基线表现好**：Chunk 级 FAISS 索引，语义检索已经足够。基于证据的结构化回答
- **Langchain 工具调用**：ReAct 循环，工具调用率 100%，证据覆盖率 115%，消除格式错误
- **结构化输出解析器**：种子回退工具调用文本，支持多种格式，容错解析，不会因工具调用失败崩溃
- **结构化评测指标**：5 维度评分设计，区分相关性、可操作性、证据质量等维度
- **成本与延迟**：Full Agent 成本是 RAG 的 2.3 倍，延迟是 2.5 倍，权衡值得吗？
- **Qwen 零样本**：Grade 5 级别，准确率 56.14%，解析失败率 0%，指令遵循能力强，无需工具调用即可输出结构化数据
- **思维链 vs 微调**：21 个错误样本中，思维链可以修复部分格式问题，但无法修复语义错误。微调模型在这两个维度上表现更稳定
- **Qwen 微调 vs 思维链**：Qwen 微调在格式合规性上表现优异，但思维链模型在语义理解上表现更好，两者互补